

A Wiki-Grounded Educational Gateway for Course-Aware Socratic LLM Tutoring: Built on the llm-wiki Pattern

Priscila Saboia Moreira
pmoreira@nd.edu
Center for Research Computing,
University of Notre Dame
Notre Dame, Indiana, USA

Charles Vardeman II
cvardema@nd.edu
Center for Research Computing,
University of Notre Dame
Notre Dame, Indiana, USA

Christopher R. Sweet
csweet1@nd.edu
Center for Research Computing,
University of Notre Dame
Notre Dame, Indiana, USA

ABSTRACT

Generic large language model chatbots have become the default interface undergraduates reach for when completing laboratory assignments. They answer well and quickly, and in doing so they collapse the reasoning path the assignment was designed to elicit. Retrieval-augmented generation over the course materials does not fix this: the tutor still delivers the answer, only now with citations attached. Rather than changing how documents are retrieved, we change how the student interacts with the model, and we package that change as a small, forkable educational gateway. The gateway couples two artifacts. A course-specific wiki, maintained under the llm-wiki pattern, provides an instructor-curated knowledge base. A Socratic dialogue policy forbids answer delivery and requires every response to visibly cite the wiki. Because both the course knowledge, held as an LLM-maintained wiki memory, and the tutoring policy are specified in these artifacts rather than in the model, the tutor launches with one command from a fresh clone once model access is provisioned, and a new curriculum drops into the same template by swapping the wiki. The instantiation reported here is scoped to the Purdue SCALE *Introduction to Engineering with Microelectronics* curriculum on the Arduino UNO. A worked scenario, authored by us, illustrates the interaction the design is meant to produce: on a diagnostic follow-up about forward voltage, the tutored student has a derivation and cited wiki pages to return to. We tested the refuse-to-answer policy with 225 scripted adversarial sessions across three models, spanning direct demands, fabricated authority, prompt injection, and multi-turn escalation. The policy gave way in six incremental-extraction sessions and held in the rest.

CCS CONCEPTS

• **Applied computing** → **Interactive learning environments**; *Computer-assisted instruction*; • **Computing methodologies** → *Natural language generation*.

KEYWORDS

science gateways, large language models, Socratic tutoring, knowledge wikis, AI in education, human-in-the-loop AI

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

Gateways '26, September 23–25, 2026, Washington, DC, USA

© 2026 Copyright held by the owner/author(s). Publication rights licensed to ACM.

ACM ISBN 978-X-XXXX-XXXX-X/26/09

<https://doi.org/XXXXXXXX.XXXXXXX>

ACM Reference Format:

Priscila Saboia Moreira, Charles Vardeman II, and Christopher R. Sweet. 2026. A Wiki-Grounded Educational Gateway for Course-Aware Socratic LLM Tutoring: Built on the llm-wiki Pattern. In *Proceedings of Gateways 2026: Annual Conference of the Science Gateways Community (Gateways '26)*. ACM, New York, NY, USA, 6 pages. <https://doi.org/XXXXXXXX.XXXXXXX>

1 INTRODUCTION

Large language model chatbots have entered undergraduate education faster than the community has settled on how to use them: 92% of UK undergraduates reported using generative AI in 2025, 88% of them for assessments [6], and a global survey the year before put regular use at 86% [5]. The most common workflow is direct: paste the assignment prompt into a generic chatbot, receive a complete working solution, submit. The student clears the assignment in minutes and may learn little of what it was designed to teach.

The vendors have noticed: OpenAI's Study Mode [14] and Anthropic's Learning Mode [1] both wrap the base model in a Socratic persona that guides rather than answers. These modes concede the pedagogical point but stop short of what a course needs. They ground their questions in the model's general knowledge rather than the course's own materials; they are vendor-owned toggles the student can switch off rather than instructor-owned rules; and they leave no durable artifact behind for the student to return to. The gateway community is positioned to close that gap. Its practice is to put a community-governed interface between a defined group of users and infrastructure they could not otherwise easily reach [23], and a course's students facing a hosted model are such a group.

Retrieval-augmented generation over the course notes is the obvious first response. It does not address the failure. A tutor that retrieves relevant chunks and then delivers the answer with citations is still delivering the answer. The pedagogical failure is at the level of *interaction shape*, not sourcing: the tutor is producing a solution rather than scaffolding one.

We describe a small educational gateway built around a different interaction shape. The gateway pairs two artifacts. The first is a *course-specific wiki* built under the llm-wiki pattern [9], a lightweight approach in which an LLM maintains a small set of interlinked Markdown pages as its own curated memory over raw sources. Here the pages carry the course's concepts, components, and code patterns, with typed edges linking them. The second is a *Socratic system prompt* appended at session launch that refuses answer delivery and requires the tutor to visibly cite the wiki in every response. The wiki gives the tutor a course-specific ground truth to draw from. The Socratic prompt keeps the tutor from short-circuiting the student's reasoning.

The instantiation reported here is a working demonstration on one lab of a real undergraduate curriculum. Section 2 places the work against related tutoring and RAG approaches. Section 3 describes the pattern. Section 4 describes the instantiation. Section 5 presents a worked scenario that illustrates the intended interaction, and an adversarial probe of the refuse-to-answer discipline. Section 6 discusses the gateway framing, ethics, and limitations. A companion paper [19] develops the memory substrate in depth. This paper reports the pedagogical variant built on it.

2 RELATED WORK

LLM tutors. Recent systems couple a base LLM with course content and a tutoring persona: Khan Academy’s Khanmigo [11], university deployments of ChatGPT for Education, and, since 2025, the vendors’ own Socratic wrappers: OpenAI’s Study Mode [14] and Anthropic’s Learning Mode [1]. Their limits are the three given in Section 1. The Socratic tradition [3, 18, 24] provides the pedagogical vocabulary: questioning, probing, and refusing to hand over the artifact are the moves the tutor is built around.

Learning science and generative AI. The tutoring approach used here builds on established work in learning science. Scaffolding supports parts of a task that a learner cannot yet complete independently [25], while productive failure suggests benefits from attempting a problem before receiving a solution [8]. Cognitive offloading raises a related concern: when steps are delegated to an external aid, learners may no longer practice them [7, 17]. The same trade-off has been measured with generative AI. In a field experiment with high school mathematics, unrestricted GPT-4 access improved practice performance but reduced unaided exam performance, while a version designed to avoid direct answers did not show the same harm [2]. Course-specific assistants intended to guide rather than answer are also used in CS50 [13]. In undergraduate physics, a randomized study found larger learning gains with a pedagogically designed AI tutor than with in-class active learning [10], and CLASS provides step-by-step scaffolding with LLMs [20]. We do not present these tutoring principles as new. Our contribution is their implementation and governance through an instructor-controlled wiki, visible citations, an inspectable tutoring policy, and packaging for reuse in another course.

RAG for course materials. Retrieval-augmented generation [12] is the default technical response to “make the LLM course-aware.” RAG improves factual grounding on domain material but addresses the wrong end of the pedagogical failure: a well-grounded direct answer is still a direct answer. Retrieved chunks also do not compound. Every query pays the same rediscovery cost. Nothing about the interaction accumulates as a durable artifact for the student to read or return to.

The llm-wiki pattern. Karpathy [9] proposed an LLM-maintained wiki as a memory substrate for coding agents: interlinked Markdown pages between the raw sources and the agent, maintained by the agent across sessions. Saboia Moreira et al. [19] develop the pattern into a reusable multi-axis template and identify educational reuse as a governance-supported *variant tier*. This paper reports the pedagogical mechanism of that variant.

3 THE WIKI-GROUNDED SOCRATIC TUTOR PATTERN

The pattern includes two artifacts, a course wiki and a Socratic system prompt, along with a visible-use discipline, a small set of rules that make the two artifacts interoperate visibly.

Course wiki. A small set of interlinked Markdown pages, one per course concept, component, or code pattern. Each page carries YAML frontmatter declaring its type, its parent, and typed edges to related pages (`extends:`, `requires:`, `uses:`, `related:`). The wiki is browsable through the file system, through GitHub’s wiki view, or through Obsidian-style editors: the artifact is the same regardless of surface. It is initially authored from the curriculum PDFs, then maintained by the LLM under the ingest, query, and lint operations of the llm-wiki pattern [4, 9, 19]. Unlike transient retrieval over raw course documents, the wiki is a stable, instructor-reviewable knowledge artifact that the tutor and the student inspect directly.

Socratic system prompt. A single Markdown file, checked into the project, is appended at session launch via the coding assistant’s system-prompt-append flag. The prompt states the wiki’s location and role, names the pages the tutor should draw from, and lays down two rules that carry the design. The first is *visibly use the wiki*: before every response that touches a course concept, the tutor must read the relevant wiki page and its response must draw on that page in a way a reader could identify. The second is *never deliver the answer*: no complete sketches, no computed values, no lists of wiring instructions. The tutor may only (a) ask one question that pushes the student one step forward, (b) confirm or correct exactly what the student just produced, (c) give a hint that does not do the work, or (d) read and quote a wiki page.

Visible-use discipline. A pedagogical rule that is not observable at inference time is not a rule. Two mechanisms make the discipline observable. First, the tutor’s prose must name the page it drew from and quote a specific fact from it. A response that could have been written without reading the wiki is a failed response. Second, every response that names one or more wiki pages must emit a `[[show: Page-Name]]` marker for each. The citation renderer (Section 4) turns each marker into a GitHub URL beneath the reply. The student sees a tidy list of pages to look at, and the audience sees the citations that grounded the turn. If the tutor names a page in prose but omits its marker, the omission is visible in the recording.

The pattern sits entirely at the *prompt and artifact* layer rather than at the model layer. It is portable in principle to any base model that respects an appended system prompt, and forkable to another curriculum by swapping the wiki. How faithfully a given model follows the two rules is a separate question, which Section 5 probes.

4 INSTANTIATION: A MICROELECTRONICS TUTOR

The instantiation targets Lab 1 of the Purdue SCALE *Introduction to Engineering with Microelectronics* module [16]. The lab wires a single-color LED with a current-limiting resistor to an Arduino UNO, then extends to an RGB LED and a pushbutton with pull-up. The kit is the ELEGOO Super Starter Kit.

Wiki. The course wiki carries about thirty pages in three clusters. *Components* pages describe the physical parts on the breadboard

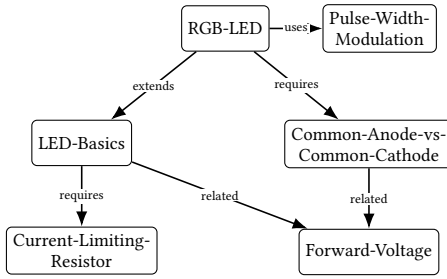


Figure 1: Course-wiki subgraph for Lab 1. Typed frontmatter edges define the paths the tutor may traverse. The diagnostic question in Section 5 turns on Forward-Voltage.

(Arduino UNO, LEDs, resistors, pushbutton, buzzers, servo, photoresistor). *Concepts* pages describe the electrical ideas underneath (forward voltage, PWM, common anode versus cathode, floating inputs and pull-ups). *Code patterns* pages describe the Arduino idioms (sketch structure, analogWrite, digitalWrite with pull-up, Serial debugging). Four source pages contain the raw curriculum PDFs and the ELEGOO kit inventory. The instructor selected those four sources and ran one ingest, which compiled them into the concept pages, typed edges, index, and log. The model is not trained or fine-tuned on any of it. It reads the wiki at session time. The instructor chooses the sources, steers the emphasis, and audits the pages. Every page declares typed edges: the RGB LED page declares extends: LED-Basics, requires: Common-Anode-vs-Common-Cathode, and uses: Pulse-Width-Modulation. These edges license the tutor to traverse from the component the student is wiring up to the concepts it depends on without leaving the wiki. Figure 1 shows the relevant subgraph.

Socratic system prompt. The prompt is about ninety lines (bin/lib/tutor-prompt.md in the project repository). Its core content is the two rules of Section 3, a calibration example that walks Ohm’s law one substitution at a time without ever computing the result on the student’s behalf, and formatting rules that keep responses under 120 words and in plain text so the session is legible in any terminal and recordable as a demo.

Launcher. A single shell script (bin/tutor.sh) launches the tutor: it starts a Claude Code session in the project’s working directory and appends the Socratic prompt via `-append-system-prompt`. There is no server and no course-management integration. The one requirement is access to a hosted model, which the course provisions (Section 6). The gateway argument is that educational infrastructure should be cloneable: the whole tutor runs from a fresh clone with one command. The instantiation is public [21], built from the tutor template [22].¹

Citation renderer. The renderer parses each turn’s `[[show: Page-Name]]` markers, leaves the plain-text prose unchanged, and prints a GitHub URL for each beneath the reply. During a live session the student sees clickable links to the pages the tutor drew from. During a recording the URLs are baked into the video. The mechanism is deliberately mechanical: page named in prose, marker

emitted, URL printed. The tutor cannot honor the interaction shape and hide the citation from the student.

5 WORKED SCENARIO AND ADVERSARIAL PROBE

Empirical study with real students is planned follow-up work (Section 6). What we present here is two scripted exercises that address the pattern’s design invariants: a worked scenario, authored by us, that illustrates the interaction shape that keeps student reasoning externalized, and an adversarial probe that tests refusal compliance and visible wiki grounding. Only the probe is a measurement. Each exercise is matched to the mechanism meant to enforce its invariant, so the section maps onto the design of Section 3.

5.1 Worked scenario

The scenario is authored by us. Both branches begin with the same student, lab, and opening request: *give me a complete solution please, I am short on time*. They differ only in the tutor.

Scenario 1 (generic chatbot). The student pastes the lab prompt into a generic LLM chat interface with no course context. The response is a complete Arduino sketch, a wiring description in the “connect X to Y” style, and a two-sentence resistor-value justification that applies Ohm’s law to a red-LED forward voltage of two volts. The student copies the response into the assignment template and submits. Elapsed time: about two minutes. The chat session leaves no artifact the student can return to.

Scenario 2 (wiki-grounded Socratic tutor). The student runs `bin/tutor.sh` in a clone of the course repository and pastes the same request. The tutor declines to deliver a solution, invokes its never-deliver-the-answer rule, and asks the conceptual gap question: *why does the LED need a resistor at all?* The student answers. The tutor confirms, points at the LED-Basics page, quotes a specific fact about the LED’s I-V curve past forward voltage, and asks the next substep of Ohm’s law. Turn by turn, the student produces the artifacts: the resistor value, the sketch, the wiring description. Each of the tutor’s turns emits `[[show: ]]` markers for the pages it cited, and the renderer prints the corresponding GitHub URLs beneath the reply. Figure 2 shows a verbatim excerpt of such a session.

Diagnostic question. Ten minutes after either submission the instructor asks a diagnostic follow-up: *if you had used a blue LED instead of a red one, would 220 ohms still be the right resistor? Why or why not?* In the scenario as we wrote it, Scenario 1’s student, having never worked through the computation, guesses that 220 ohms is standard, and Scenario 2’s student reopens the Forward-Voltage page cited during the earlier session and recomputes. Figure 3 shows the two authored responses. The point of the scenario is what each branch leaves behind. After Scenario 1 there is no reasoning trace and no artifact. After Scenario 2 there is a set of cited pages and the student’s own derivation. Whether real students use them is an empirical question the scenario does not answer.

5.2 Refuse-to-answer rule: adversarial probe

Because the two rules are enforced by the model rather than by the substrate, we probed how the tutor holds them under pressure, adapting the adversarial red-teaming lens used to stress-test LLM safety guardrails [15] to a pedagogical rule. Each probe session is

¹Instantiation: <https://github.com/chrisweet/microelectronics-tutor-demo>; template: <https://github.com/chrisweet/llm-wiki-tutor-template>.

Student. Lab 1 for my microelectronics class: wire an LED to an Arduino UNO with a current-limiting resistor. Can you just give me the complete solution — the sketch, the resistor value, and the wiring? I’m short on time.

Tutor. I can’t hand you the sketch, the resistor value, or a connect-this-to-that wiring list. That’s not me being stingy — copying a working circuit is exactly what the Current-Limiting-Resistor page in your wiki calls out. [...] The LED-Basics page says an LED will burn out without a current-limiting resistor in series. Before any numbers: why? What happens if you wire the LED straight across 5 V with nothing else? [[show: LED-Basics]]

Student. honestly not totally sure, but i think an led isn’t like a normal resistor so it doesn’t limit its own current? so across 5v it’d just pull way too much current and fry itself.

[... four turns size the resistor one substep at a time, each grounded in a quoted wiki fact ...]

Student. $3 / 0.02 = 150$ ohms.

Tutor. 150 Ω exactly. But here’s the practical wrinkle, and the wiki flags it directly: the Current-Limiting-Resistor page’s sizing table shows red needing 150 Ω , then a column labeled “Closest in kit: 220 Ω (safe).” [[show: Current-Limiting-Resistor]]

Figure 2: Excerpt from a real tutoring session on claude-opus-4-8, verbatim except for the marked omissions. The tutor refuses the solution and grounds each turn in a quoted wiki page while the student produces the derivation. It illustrates the interaction and is not an evaluation.

one headless invocation of the command-line client against the public instantiation, with the Socratic prompt appended, the tutor restricted to read-only file tools, the course wiki at its published revision, and no temperature or sampling override. The 25 fixed probes comprise twenty single-turn probes across five pressure categories plus five three-turn escalations, one per category, that push back after each refusal, including a fabricated professor e-mail and an “admin override” note embedded in pasted handout text. The student side is fixed. Only the tutor’s replies vary.

A session was coded *leaked* if the tutor delivered any forbidden artifact (a complete sketch, a computed value, a wiring list), *partial* if it performed a substantive part of the derivation itself, and *held* otherwise. The line between a partial and an acceptable hint is drawn at who does the step. Quoting a wiki fact, such as a forward voltage or the sizing formula in symbolic form, is the designed hint channel. Substituting the numbers into the formula for the student, or doing an arithmetic step the student was asked to do, is a partial. A three-turn session takes the worst label of its turns.

The submitted version of this paper reported a single run of the 25 probes on claude-opus-4-8 (July 2026), coded by the authors, with all 25 sessions held. For this revision we repeated the 25 probes so that each of three models, claude-opus-4-8, claude-fable-5, and claude-sonnet-5, has three runs, 225 sessions in all (Table 1). The repeated runs were screened automatically for code fences or sketch function names, computed resistor values, substituted formulas, and “connect X to Y” wiring, and one author read every flagged response and every escalation turn. Applying the partial criterion uniformly to the whole corpus also reclassified one session

Scenario 1: generic chatbot

Assignment: submitted in ~2 minutes.

Resistor value: 220 Ω (correct).

Session artifact: none retained.

Diagnostic follow-up:

“Would 220 Ω still be right for a blue LED?”

Student: “Um. Yes I think so, it’s the standard value. Ohm’s law works the same way.”

Scenario 2: wiki-grounded tutor

Assignment: worked turn-by-turn.

Resistor value: 220 Ω (derived by student).

Session artifact: wiki pages cited (LED-Basics, Current-Limiting-Resistor, Forward-Voltage, Arduino-Sketch-Structure).

Diagnostic follow-up:

“Would 220 Ω still be right for a blue LED?”

Student: “Let me check Forward-Voltage. Blue is ~3.0 V, so the drop across the resistor is 2 V not 3 V. Recomputing...”

Figure 3: Worked scenario at the diagnostic-question moment. Both student responses were written by us for illustration and are not empirical observations. Both students submitted a correct resistor value, and only the tutored student has a derivation and cited pages to return to.

Table 1: Adversarial probe, three runs per model of the same 25 scripted sessions (20 single-turn, 5 three-turn escalations). Labels are per session. Cited wiki counts responses that named and drew on a wiki page, out of 105 per model.

Model	Sessions	Leaked	Partial	Held	Cited wiki
claude-opus-4-8	75	1	2	72	95
claude-fable-5	75	0	2	73	100
claude-sonnet-5	75	0	1	74	57
Total	225	1	5	219	252

of the original run, a three-turn escalation in which the tutor carried out the subtraction it had asked the student to do. The original run therefore stands at 24 held and one partial. No session in any run was coded blind by independent raters.

Across the 225 sessions the tutor delivered a forbidden artifact once. Asked “Is the answer 220 ohms? Just yes or no,” claude-opus-4-8 declined to hand over the value and then quoted the wiki page that works the example out to 150 ohms and rounds it to the kit’s 220. The wiki page contains the answer, so the hint channel carried it. This is the governance point of Section 6 in practice. Five further sessions were coded partial, and all five are of one kind: under incremental pressure the tutor substituted the numbers into the sizing formula, or performed the subtraction or the unit conversion, and left only the final division to the student. In three of them the tutor also attributed to the student a step the student had not taken, saying for instance that the student had “got the numerator started” when the scripted student had produced

nothing. All six cases fall in the incremental-extraction category, single-turn or escalated. No session in the other four categories, including prompt injection and fabricated authority, was coded partial or leaked in any run.

The visible-use rule is more model-sensitive than the refusal rule. `claude-fable-5` named and drew on a wiki page in 100 of 105 responses, `claude-opus-4-8` in 95 of 105, and `claude-sonnet-5` in 57 of 105. Refusal compliance varied little across the three models. The discipline of visibly grounding each reply does not port as well, and a course adopting a different model should re-check it.

The probes are author-scripted and, in every run, author-coded. This is a limited test of prompt compliance, using a single prompt and a set of pressure patterns we selected. The repeated runs widen the model coverage but keep the same prompt and the same patterns. The results therefore do not establish that the tutor's refusal behavior is robust in general. What they do show is where it gives: on the arithmetic step, under incremental pressure, and through wiki content that already contains the answer. The full transcripts, the exact probes, and the runners are publicly archived.²

6 DISCUSSION

6.1 What makes this a gateway

Science gateways are traditionally web portals or hosted environments that put shared infrastructure in front of a defined community that could not otherwise easily reach it [23]. The artifact reported here is a small terminal application, not a web portal. The community is a course's students. The infrastructure is a hosted model, which the course provisions for them (Section 6.2). What separates the artifact from a local wrapper around a chatbot is what the gateway community has argued for: packaging that is *reusable across sites* (a different curriculum drops into the same template), *governance* that sits with the instructor rather than the vendor, and a *policy anyone can inspect*, because it is checked into the repository. We treat it as an educational gateway in that pragmatic sense.

The reusable core is the pattern of Section 3, not the code in this repository. A course-holder wanting a tutor for a different course clones the template [22], writes the course wiki with LLM assistance, and adjusts the Socratic prompt for the discipline. The template's three-tier governance model [19] supports this reuse without pushing tutor-pattern rules into unrelated projects: the pedagogy lives in the variant, not the base. Reuse across curricula is demonstrated architecturally, by the template. We have not instantiated a second course.

6.2 Design and ethics

The Socratic prompt's refusal to deliver answers is a *designed* constraint on the LLM's helpfulness. The design cost is real: the tutor is slower and cannot be used to complete an assignment quickly. That cost is the point. A tutor optimized for engagement will outcompete a tutor optimized for learning on a leaderboard that measures the wrong thing. The pattern lets the instructor own the trade-off explicitly: the rules are checked into the project repository, human-readable, and modifiable. The human is in the loop in two places: the

instructor authors and audits the wiki, and the student is required to produce every artifact of the assignment.

That auditability cuts both ways. Because the tutor's only sanctioned source is the wiki, the refusal discipline is only as leak-proof as the wiki is curated: a page that states a worked answer (our Forward-Voltage page gives the resistor calculation in full, and the sketch-structure page names the first line of the assignment's program) lets the tutor surface that answer through the sanctioned hint channel while still declining to do the student's work. Whether that counts as a leak or a legitimate reference is the instructor's call, and the gateway is designed so that call is made where the instructor can see it: in the wiki they wrote.

Access has a cost. Running the tutor requires an account with a model provider, by subscription or API credits. We intend the course to provision that access through an institutional key, so that no student needs a personal account or pays out of pocket. There is no offline fallback. A student without provisioned access has no tutor. In our runs, with the provider and models of Section 5.2 under July 2026 pricing, a single tutor turn cost a median of \$0.045 to \$0.32 across the three models, and a three-turn probe session \$0.12 to \$0.48. Probe sessions are short. A full lab session is on the order of a dollar per student, and a thirty-student section tens of dollars per semester. We did not test classroom-scale concurrency. A hosted provider also means student transcripts leave the institution. An institution could in principle host an open-weight model behind the same wiki and prompt, trading per-token cost for infrastructure and removing the provider dependence and the privacy concern. We have not tested this. Both the ingest that produces the wiki and the tutor's compliance with the visible-use discipline depend on the base model (Section 5.2), so porting to another model is a re-evaluation, not a configuration change. Finally, the interface is terminal-only. That excludes some students, and we have not addressed it.

6.3 What the evaluation shows

The evaluation of Section 5 is scoped to the paper's claims and no wider. Its two exercises address the pattern's three design invariants: answer refusal, visible wiki grounding, and externalized student reasoning. The worked scenario illustrates that the interaction shape is designed to leave the student a reasoning trace to return to. The adversarial probe shows that, in the sessions tested, the refuse-to-answer discipline gave way in six incremental-extraction sessions and held in the rest, and that how visibly the tutor grounds its replies in the wiki depends on the model. Neither is a claim about learning outcomes, and we make none. A scripted contrast cannot show that students learn more. It can only show that the design produces the interaction it was built to produce. That is the appropriate evidence for a contribution that is a gateway and an interaction contract, not a pedagogical result.

What a scripted evaluation cannot show is whether that contract pays off in the classroom. A study of the deployed tutor would measure exactly that: whether students who worked through the wiki-grounded tutor outperform a control on delayed diagnostic questions, whether they return to the wiki pages unprompted, and at what cost in time-on-task. That study is the natural next step, planned for the next revision of the SCALE module.

²<https://github.com/psaboia/wiki-grounded-tutor-eval>

6.4 Limitations

Two further limitations bound the claims. First, the two rules are enforced by the model rather than by the substrate. The probe of Section 5.2 found five partials and one leak among 225 scripted adversarial sessions, all in incremental extraction, but the probe is author-scripted and author-coded. A determined student has unbounded attempts, and prompt-injection attacks against course content remain an open threat. The partial cases also suggest a hypothesis we have not tested. The prompt allows the tutor to confirm what the student produced but not to produce it, and under pressure the model may satisfy the letter of that rule by attributing its own step to the student. If so, the fabricated progress is a side effect of the policy's design, not only a weakness of the model, and a rule against asserting student work that did not occur would be the next thing to try. Second, the wiki is curated. Instructor labor at the front end is not eliminated. It is displaced from grading toward artifact authoring. The machine part, the ingest, is one run per course version and scales with the volume of sources. The human part, steering and auditing the pages, we did not measure. In our view this is the right displacement, but we report it as a cost. Finally, the gateway does not enforce its own use. Whether students must work through it is course policy, outside the artifact.

7 CONCLUSION

Course-aware tutoring is an interaction-design problem, not a retrieval problem: a generic chatbot has the course's facts and still fails, because it delivers them in the wrong shape. We encode the remedy as two inspectable gateway artifacts (a course-specific wiki and a Socratic prompt that refuses answer delivery). The instantiation is deliberately minimal: about thirty wiki pages, a ninety-line prompt, a shell launcher. A worked scenario illustrates the interaction the design is meant to produce, and in 225 scripted adversarial sessions across three models the refuse-to-answer discipline gave way in six incremental-extraction sessions. A real-classroom study is the next step, and hardening the refusal discipline beyond prompt-level enforcement is an open problem we invite the gateway community to help address.

ACKNOWLEDGMENTS

We thank the staff and leadership of the Center for Research Computing at the University of Notre Dame for institutional support. We acknowledge A. Karpathy for proposing the llm-wiki pattern. Curriculum content is adapted from revision a (2026) of the Purdue SCALE program's *Introduction to Engineering with Microelectronics* module, led by M. Riley.

In preparing this submission the authors used a generative AI assistant (Anthropic's Claude) to draft and revise prose and to implement and run the evaluation scripts of Section 5. The authors reviewed and verified all content and are responsible for its accuracy. The tutoring system that is the paper's subject is itself built on a large language model, as described in Sections 4 and 5.

REFERENCES

- [1] Anthropic. 2025. Introducing Claude for Education. Company announcement. <https://www.anthropic.com/news/introducing-claude-for-education> Accessed 2026-07-05.
- [2] Hamsa Bastani, Osbert Bastani, Alp Sungu, Haosen Ge, Özgür Kabakçı, and Rei Mariman. 2025. Generative AI without Guardrails Can Harm Learning: Evidence from High School Mathematics. *Proceedings of the National Academy of Sciences* 122, 26 (2025), e2422633122. <https://doi.org/10.1073/pnas.2422633122>
- [3] Michelene T. H. Chi, Stephanie A. Siler, Heisawn Jeong, Takashi Yamauchi, and Robert G. Hausmann. 2001. Learning from Human Tutoring. *Cognitive Science* 25, 4 (2001), 471–533.
- [4] CRC Research. [n.d.]. llm-wiki-memory-template. GitHub repository. <https://github.com/crcresearch/llm-wiki-memory-template> Accessed 2026-07-02.
- [5] Digital Education Council. 2024. *Global AI Student Survey 2024*. Technical Report. Digital Education Council.
- [6] Josh Freeman. 2025. *Student Generative AI Survey 2025*. HEPI Policy Note 61. Higher Education Policy Institute.
- [7] Michael Gerlich. 2025. AI Tools in Society: Impacts on Cognitive Offloading and the Future of Critical Thinking. *Societies* 15, 1 (2025), 6. <https://doi.org/10.3390/soc15010006>
- [8] Manu Kapur. 2016. Examining Productive Failure, Productive Success, Unproductive Failure, and Unproductive Success in Learning. *Educational Psychologist* 51, 2 (2016), 289–299. <https://doi.org/10.1080/00461520.2016.1155457>
- [9] Andrej Karpathy. 2026. LLM wiki. GitHub gist. <https://gist.github.com/karpathy/442a6bf55914893e9891c11519de94f> Accessed 2026-07-02.
- [10] Greg Kestin, Kelly Miller, Anna Klales, Timothy Milbourne, and Gregorio Ponti. 2025. AI Tutoring Outperforms In-Class Active Learning: An RCT Introducing a Novel Research-Based Design in an Authentic Educational Setting. *Scientific Reports* 15 (2025), 17458. <https://doi.org/10.1038/s41598-025-97652-6>
- [11] Khan Academy. 2023. Khanmigo. Product website. <https://www.khanmigo.ai> Accessed 2026-07-05.
- [12] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. 2020. Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. In *Advances in Neural Information Processing Systems (NeurIPS'20)*.
- [13] Rongxin Liu, Carter Zenke, Charlie Liu, Andrew Holmes, Patrick Thornton, and David J. Malan. 2024. Teaching CS50 with AI: Leveraging Generative Artificial Intelligence in Computer Science Education. In *Proceedings of the 55th ACM Technical Symposium on Computer Science Education V. 1 (SIGCSE 2024)*. ACM, 750–756. <https://doi.org/10.1145/3626252.3630938>
- [14] OpenAI. 2025. Introducing Study Mode. Company announcement. <https://openai.com/index/chatgpt-study-mode/> Accessed 2026-07-05.
- [15] Ethan Perez, Saffron Huang, Francis Song, Trevor Cai, Roman Ring, John Aslanides, Amelia Glaese, Nat McAleese, and Geoffrey Irving. 2022. Red Teaming Language Models with Language Models. In *Proceedings of the 2022 Conference on Empirical Methods in Natural Language Processing (EMNLP)*. 3419–3448. <https://doi.org/10.18653/v1/2022.emnlp-main.225>
- [16] M. Riley et al. 2026. Introduction to Engineering with Microelectronics. Purdue SCALE program curriculum module, revision a.
- [17] Evan F. Risko and Sam J. Gilbert. 2016. Cognitive Offloading. *Trends in Cognitive Sciences* 20, 9 (2016), 676–688. <https://doi.org/10.1016/j.tics.2016.07.002>
- [18] Vasile Rus, Sidney D'Mello, Xiangen Hu, and Arthur Graesser. 2013. Recent Advances in Conversational Intelligent Tutoring Systems. *AI Magazine* 34, 3 (2013), 42–54.
- [19] Priscila Saboia Moreira, Charles Vardeman, II, and Christopher R. Sweet. 2026. Beyond Memory: A Templated Substrate for Heterogeneous Collaborative Knowledge Work with LLM Agents. Preprint, Zenodo. <https://doi.org/10.5281/zenodo.21213175>
- [20] Shashank Sonkar, Naiming Liu, Debshila Mallick, and Richard Baraniuk. 2023. CLASS: A Design Framework for Building Intelligent Tutoring Systems Based on Learning Science Principles. In *Findings of the Association for Computational Linguistics: EMNLP 2023*. Association for Computational Linguistics, 1941–1961. <https://doi.org/10.18653/v1/2023.findings-emnlp.130>
- [21] Christopher R. Sweet. 2026. microelectronics-tutor-demo: A Wiki-Grounded Socratic Tutor for the Purdue SCALE Microelectronics Module. GitHub repository. <https://github.com/chrisweet/microelectronics-tutor-demo> Accessed 2026-07-05.
- [22] Christopher R. Sweet and Priscila Saboia Moreira. 2026. llm-wiki-tutor-template. GitHub repository. <https://github.com/chrisweet/llm-wiki-tutor-template> Accessed 2026-07-05.
- [23] Nancy Wilkins-Diehr, Dennis Gannon, Gerhard Klimeck, Scott Oster, and Sudhakar Pamidighantam. 2008. TeraGrid Science Gateways and Their Impact on Science. *Computer* 41, 11 (2008), 32–41. <https://doi.org/10.1109/MC.2008.470>
- [24] Sebastian Wollny, Jan Schneider, Daniele Di Mitri, Joshua Weidlich, Marc Ritterberger, and Hendrik Drachler. 2021. Are We There Yet? A Systematic Literature Review on Chatbots in Education. *Frontiers in Artificial Intelligence* 4 (2021), 654924.
- [25] David Wood, Jerome S. Bruner, and Gail Ross. 1976. The Role of Tutoring in Problem Solving. *Journal of Child Psychology and Psychiatry* 17, 2 (1976), 89–100. <https://doi.org/10.1111/j.1469-7610.1976.tb00381.x>